Tensor Product Model Transformation

Theory, Hull Decompositions, and Application to State Trajectories

Notes

Contents

1 Motivation and Overview **2**

2 Data Matrix and Problem Setup **2**

3 Step 1 — Singular Value Decomposition and Rank Truncation **3**
- 3.1 Full SVD . . . 3
- 3.2 Rank Truncation . . . 3
- 3.3 Why SVD First? . . . 3

4 Step 2 — Inner Centering and Secondary SVD **3**

5 Step 3 — SNNN Hull Decomposition **4**
- 5.1 Definition . . . 4
- 5.2 Construction . . . 4
- 5.3 Vertex Matrix . . . 5

6 Step 4 — CNO Hull Decomposition **5**
- 6.1 Motivation . . . 5
- 6.2 Polar Coordinate Parameterisation of the Simplex . . . 5
- 6.3 Cost Function . . . 6
- 6.4 Optimisation Strategy . . . 6
- 6.5 Back-Transformation . . . 6

7 Step 5 — IRNO Hull Decomposition **6**
- 7.1 Motivation . . . 6
- 7.2 Construction . . . 7

8 Vertex Systems and Reconstruction **7**
- 8.1 Vertex Systems . . . 7
- 8.2 Image-Space Projection . . . 7
- 8.3 Reconstruction Error . . . 8

9 Full Algorithm Summary **8**

10 Connection to LMI-Based Controller Synthesis **8**
- 10.1 Effect of Hull Type on Controller Design . . . 8

1 Motivation and Overview

The *Tensor Product (TP) model transformation* [1] is a framework for representing a parameter-varying system or a data matrix as a convex combination of a finite set of *vertex systems*. Given a trajectory matrix $\mathbf{F} \in \mathbb{R}^{N \times n}$, the goal is to find

$$\mathbf{F} \approx \mathbf{W}\,\mathbf{S}, \qquad \mathbf{W} \in \mathbb{R}^{N \times R'}, \quad \mathbf{S} \in \mathbb{R}^{R' \times n}, \tag{1}$$

where the rows of $\mathbf{W}$ (the *weighting functions*) satisfy convexity constraints, and the rows of $\mathbf{S}$ are the *vertex systems*—the prototype states whose convex combination reproduces $\mathbf{F}$.

The choice of convexity constraint on $\mathbf{W}$ defines the *hull type*:

- **SNNN** (Sum Normalized, Non-Negative): each row of $\mathbf{W}$ sums to one and is entry-wise non-negative.
- **CNO** (Close to Normal): minimises the largest weight entry, making the distribution of vertex influence as balanced as possible.
- **IRNO** (Inverted Reciprocal Normal): an alternative normalization that equalises the reciprocal of the weights.

The TP form (1) is foundational for LMI (Linear Matrix Inequality) based controller synthesis: once $\mathbf{F}$ is expressed as a convex combination of vertex systems, stability and performance conditions reduce to finite sets of LMIs evaluated at the vertices.

2 Data Matrix and Problem Setup

Let $\{\mathbf{x}_k\}_{k=0}^{T}$ be the state trajectory of a discrete-time system, with $\mathbf{x}_k \in \mathbb{R}^n$. Stack the trajectory as rows of

$$\mathbf{F} \;=\; \begin{pmatrix} \mathbf{x}_0^{\mathsf{T}} \\ \mathbf{x}_1^{\mathsf{T}} \\ \vdots \\ \mathbf{x}_T^{\mathsf{T}} \end{pmatrix} \;\in\; \mathbb{R}^{N \times n}, \qquad N = T + 1. \tag{2}$$

The TP transformation seeks a factorisation $\mathbf{F} = \mathbf{WS}$ (exact or approximate) with R' vertex systems, where the columns of $\mathbf{W}$ define how each vertex contributes at each time step.

Remark 2.1. In the general TP framework, $\mathbf{F}$ need not come from a time series; it can be any sampled representation of a parameter-varying function. Here we specialise to state trajectories, which is the setting of the accompanying code.

3 Step 1 — Singular Value Decomposition and Rank Truncation

3.1 Full SVD

Compute the (economy) SVD of $\mathbf{F}$:

$$\mathbf{F} = \mathbf{U}\,\mathbf{\Sigma}\,\mathbf{V}^{\mathsf{T}}, \tag{3}$$

where

- $\mathbf{U} \in \mathbb{R}^{N\times n}$ has orthonormal columns ($\mathbf{U}^{\mathsf{T}}\mathbf{U} = \mathbf{I}_n$),
- $\mathbf{\Sigma} = \mathrm{diag}(\sigma_1, \ldots, \sigma_n)$ with $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_n \geq 0$,
- $\mathbf{V} \in \mathbb{R}^{n\times n}$ is orthogonal.

3.2 Rank Truncation

Choose rank R as the number of singular values exceeding a relative threshold ε:

$$R = \#\big\{i : \sigma_i > \varepsilon\,\sigma_1\big\}, \qquad R \geq 2. \tag{4}$$

Typical choice: $\varepsilon = 0.01$ (keep components carrying more than 1% of the leading singular value). Define the truncated factor

$$\mathbf{U}_R = \mathbf{U}_{:,\,1:R} \in \mathbb{R}^{N\times R}. \tag{5}$$

The low-rank approximation is

$$\mathbf{F} \approx \mathbf{U}_R\,\mathbf{\Sigma}_R\,\mathbf{V}_R^{\mathsf{T}} \;=:\; \mathbf{U}_R\,\mathbf{S}_{\mathrm{SVD}}, \tag{6}$$

where $\mathbf{S}_{\mathrm{SVD}} = \mathbf{\Sigma}_R\mathbf{V}_R^{\mathsf{T}} \in \mathbb{R}^{R\times n}$ is the "raw" vertex matrix in the SVD basis.

3.3 Why SVD First?

The columns of $\mathbf{U}_R$ form an *orthonormal* basis for the R-dimensional subspace that best explains variance in $\mathbf{F}$. Working in this reduced space makes the subsequent hull decomposition computationally tractable and numerically well-conditioned.

4 Step 2 — Inner Centering and Secondary SVD

Before applying a hull algorithm, the code centers and re-orthogonalises $\mathbf{U}_R$ to remove any mean offset. Given input $\mathbf{U} \in \mathbb{R}^{N\times R}$:

1. Compute the column-wise mean: $\hat{\boldsymbol{u}} = \frac{1}{N}\mathbf{U}^{\mathsf{T}}\mathbf{1}_N \in \mathbb{R}^R$.
2. Center: $\tilde{\mathbf{U}} = \mathbf{U} - \mathbf{1}_N\hat{\boldsymbol{u}}^{\mathsf{T}}$.
3. Economy SVD of the centered matrix: $\tilde{\mathbf{U}} = \mathbf{U}_u\,\mathbf{S}_u\,\mathbf{V}_u^{\mathsf{T}}$.
4. Retain n_s columns where $(\mathbf{S}_u)_{ii} > 10^{-5}$: $\mathbf{U}_u \leftarrow \mathbf{U}_u^{:,\,1:n_s}$.

5. If $n_s < R$ (the centered matrix has lower rank than R), replace $\mathbf{U}$ with $\mathbf{U}_u$ for the hull step.

After the hull is computed in the centered-and-compressed space, the vertex matrix is mapped back via

$$\mathbf{S} \leftarrow \mathbf{S}\,\mathbf{S}_u\,\mathbf{V}_u^{\mathsf{T}} + \mathbf{1}_{R'}\hat{\boldsymbol{u}}^{\mathsf{T}}. \tag{7}$$

Remark 4.1. In practice, when $\mathbf{U}$ is already the top-R left singular vectors of $\mathbf{F}$ (as in the code's main analysis path), its columns are nearly zero-mean and the centering step has negligible effect.

5 Step 3 — SNNN Hull Decomposition

5.1 Definition

Definition 5.1 (SNNN weighting functions)**.** A matrix $\mathbf{W} \in \mathbb{R}^{N\times R'}$ is *SNNN* if

$$w_{ij} \geq 0 \;\; \forall\, i,j \qquad \text{and} \qquad \sum_{j=1}^{R'} w_{ij} = 1 \;\; \forall\, i. \tag{8}$$

Each row is a probability vector; the rows of $\mathbf{S}$ are the *vertices* of the enclosing simplex.

5.2 Construction

Given $\mathbf{U} \in \mathbb{R}^{N\times R}$ (the post-centering left singular vectors), construct the augmented matrix $\tilde{\mathbf{U}} \in \mathbb{R}^{N\times(R+1)}$:

$$\tilde{U}_{i,\,R+1} = 1 - \sum_{j=1}^{R} U_{ij}, \qquad \tilde{\mathbf{U}} = \big[\,\mathbf{U} \;\big|\; \mathbf{1}_N - \mathbf{U}\mathbf{1}_R\,\big]. \tag{9}$$

By construction, every row of $\tilde{\mathbf{U}}$ sums to one. Let $n_a = R + 1$ denote the number of augmented columns. Define

$$u_{\min} = \min\!\left(0,\; \min_{i,j} \tilde{U}_{ij}\right), \qquad u_{\max} = \max\!\left(\tfrac{2}{n_a},\; \max_{i,j} \tilde{U}_{ij}\right). \tag{10}$$

The floor 0 on $u_{\min}$ and the floor $2/n_a$ on $u_{\max}$ guarantee the denominators below are non-zero and that the resulting transformation is non-trivial.

Compute two candidate scaling constants:

$$c_1 = \frac{1}{1 - n_a\,u_{\min}}, \qquad c_2 = \frac{1}{1 - n_a\,u_{\max}}, \tag{11}$$

and choose

$$c = \begin{cases} c_2 & \text{if } c_1 < -c_2, \\ c_1 & \text{otherwise,} \end{cases} \qquad d = \frac{1-c}{n_a}. \tag{12}$$

Form the $(n_a \times n_a)$ mixing matrix

$$\mathbf{M} = d\,\mathbf{1}_{n_a}\mathbf{1}_{n_a}^{\mathsf{T}} + c\,\mathbf{I}_{n_a}, \tag{13}$$

and compute the SNNN weighting matrix

$$\mathbf{W}_{\text{SNNN}} = \tilde{\mathbf{U}}\,\mathbf{M} \in \mathbb{R}^{N\times n_a}. \tag{14}$$

Theorem 5.1 (SNNN properties of $\mathbf{W}_{\text{SNNN}}$)**.** $\mathbf{W}_{\text{SNNN}}$ *defined by* (14)–(12) *satisfies the SNNN conditions* (8)*.*

Proof. (Sum to one.) Each entry of $\mathbf{W}_{\text{SNNN}}$ satisfies

$$W_{ij} = d\sum_k \tilde{U}_{ik} + c\,\tilde{U}_{ij} = d\cdot 1 + c\,\tilde{U}_{ij},$$

because rows of $\tilde{\mathbf{U}}$ sum to one. Summing over j:

$$\sum_j W_{ij} = n_a\,d + c\sum_j \tilde{U}_{ij} = n_a\,d + c\cdot 1 = (1-c) + c = 1.\checkmark$$

(Non-negativity.) $W_{ij} = d + c\,\tilde{U}_{ij} \geq 0$ requires $d \geq -c\,\tilde{U}_{ij}$ for all i, j. The binding constraints are at $u_{\min}$ (when $c > 0$) and at $u_{\max}$ (when $c < 0$). The definitions of c_1, c_2 are exactly the values of c at which these constraints become active, and the choice (12) selects the one that yields $d + c\,u_{\min} \geq 0$ and $d + c\,u_{\max} \geq 0$ simultaneously. ✓ □

5.3 Vertex Matrix

The SNNN vertex matrix is obtained by least-squares:

$$\mathbf{S}_{\text{SNNN}} = \mathbf{W}_{\text{SNNN}}^{\dagger}\,\mathbf{F} = \arg\min_{\mathbf{S}} \|\mathbf{F} - \mathbf{W}_{\text{SNNN}}\mathbf{S}\|_F\,. \tag{15}$$

6 Step 4 — CNO Hull Decomposition

6.1 Motivation

SNNN produces a valid simplex hull but does not minimise the *spread* of the weighting functions. Large maximum weights imply that the system spends most of its time near a single vertex, reducing the utility of the multi-vertex representation. The *Close to Normal* (CNO) algorithm seeks vertex directions that make the distribution of weights as uniform as possible.

6.2 Polar Coordinate Parameterisation of the Simplex

A vertex matrix $\mathbf{v} \in \mathbb{R}^{R'\times R'}$ (rows = vertices in weight space) is parameterised via generalised polar coordinates $\mathbf{a} \in \mathbb{R}^{R'\times(R'-2)}$:

$$v_{i,\ell} = \begin{cases} \sin(a_{i,\ell})\prod_{j=1}^{\ell-1}\cos(a_{i,j}) & \ell = 1,\ldots,R'-2, \\ \prod_{j=1}^{R'-2}\cos(a_{i,j}) & \ell = R'-1, \\ 1 - \sum_{j=1}^{R'-1} v_{i,j} & \ell = R'. \end{cases} \tag{16}$$

The last column enforces the simplex sum constraint. The centre of $\mathbf{v}$ is shifted to match the centroid of the data.

6.3 Cost Function

After computing $\mathbf{U}_2 = \mathbf{U}\,\mathbf{v}^{-1}$ (the weights expressed in the vertex coordinate frame), the cost function is

$$\mathcal{L}(\mathbf{a}) = \left\| \log \max_i (\mathbf{U}_2)_{:,\cdot} \right\|_h + \lambda \left\| \log \max\big(\mathbf{U}_2[0,:],\, \mathbf{U}_2[-1,:]\big)_{R'-2:R'} \right\|_h , \tag{17}$$

where $\|\cdot\|_h$ denotes the h-norm ($h = 1$ in practice) and $\lambda > 0$ penalises large values at the boundary rows (first and last time steps). Minimising $\mathcal{L}$ makes the maximum weight across vertices as small as possible—i.e. as close to $1/R'$ as possible.

Remark 6.1. The inversion $\mathbf{U}\,\mathbf{v}^{-1}$ inside the optimisation loop is replaced in the implementation by column-wise linear solves: only column i of $\mathbf{U}\mathbf{v}^{-1}$ is needed at each inner iteration, so computing $\mathbf{c}_i = \mathbf{v}^{-1}\mathbf{e}_i$ (solving $\mathbf{v}\mathbf{c}_i = \mathbf{e}_i$) followed by $\mathbf{U}\mathbf{c}_i$ avoids the full $(N \times R') \times (R' \times R')$ matrix multiply, reducing per-evaluation cost from $O(NR'^2)$ to $O(R'^2 + NR')$.

6.4 Optimisation Strategy

The CNO optimisation uses a multi-start Nelder–Mead scheme with three phases:

1. **Random starts** (parallelised): n_{vel} independent Nelder–Mead minimisations from random initialisations of $\mathbf{a}$. The best result is retained as $\mathbf{a}^*$.
2. **Local refinement**: Repeat Nelder–Mead from $\mathbf{a}^*$ until n_{lok} consecutive non-improving iterations.
3. **Perturbation search** (parallelised): n_{zav} perturbed starts $\mathbf{a}^* + 0.1\,\epsilon\,\mathbf{z}$ (random $\mathbf{z}$), each with its own local refinement.

Default parameters: $n_{\text{vel}} = 50$, $n_{\text{zav}} = 15$, $n_{\text{lok}} = 10$.

6.5 Back-Transformation

CNO operates on the SNNN-transformed matrix $\mathbf{W}_{\text{SNNN}}$ rather than $\mathbf{U}_R$ directly. The returned vertex matrix $\mathbf{v}_2$ and weighting matrix $\mathbf{U}_2$ satisfy

$$\mathbf{W}_{\text{SNNN}} = \mathbf{U}_2\,\mathbf{v}_2, \quad \Rightarrow \quad \mathbf{W}_{\text{CNO}} = \mathbf{U}_2, \quad \mathbf{S}_{\text{CNO}} = \mathbf{v}_2\,\mathbf{S}_{\text{SNNN}}. \tag{18}$$

Thus CNO applies a linear change of vertex basis *within* the SNNN simplex. The SNNN constraints are preserved (the transform is an interior linear map of the simplex), while the maximum weight is reduced.

7 Step 5 — IRNO Hull Decomposition

7.1 Motivation

IRNO (Inverted Reciprocal Normal) seeks a scaling c such that the normalised weight matrix satisfies a complementary normalisation: the reciprocal of weights is well-conditioned. It provides an intermediate decomposition between SNNN (which may have very small weights) and CNO (which minimises the maximum weight).

7.2 Construction

IRNO also starts from $\mathbf{W}_{\text{SNNN}}$ and finds a scalar $c \in [0, \infty)$ by bisection on the function

$$\phi(c) = 1 - \min_i \sum_j u_{ij}^{(4)}(c) - \max_j u_{i^*,j}^{(4)}(c), \tag{19}$$

where $u^{(4)}$ is obtained from a chain of normalisation steps applied to $\mathbf{U}$ with parameter c:

$$\mathbf{U}^{(1)} = \mathbf{U}_0 + \frac{c-1}{R}\,(\mathbf{U}_0 \mathbf{1}_R)\mathbf{1}_R^{\mathsf{T}}, \tag{20}$$

$$\mathbf{U}^{(2)} = \mathbf{U}^{(1)} - \mathbf{1}_N \min_i(\mathbf{U}^{(1)})_j, \tag{21}$$

$$\mathbf{U}^{(3)} = \mathbf{U}^{(2)} \oslash (\mathbf{1}_N \max_i(\mathbf{U}^{(2)})_j), \tag{22}$$

$$\mathbf{U}^{(4)} = \mathbf{U}^{(3)} \big/ \max_i \sum_j U_{ij}^{(3)}, \tag{23}$$

where $\oslash$ denotes element-wise division. The scalar c^* is found by bisection on $\phi(c) = 0$ over $[0,\, c_{\max}]$, where $c_{\max}$ is doubled until $\phi(c_{\max}) < 0$.

The IRNO weighting matrix is then

$$\mathbf{W}_{\text{IRNO}} = \left[\mathbf{U}^{(4)}(c^*) \;\middle|\; \mathbf{1}_N - \mathbf{U}^{(4)}(c^*)\mathbf{1}_R\right], \tag{24}$$

and the vertex matrix follows from the transformation $\mathbf{f}_i$ that maps $\mathbf{W}_{\text{SNNN}}$ to $\mathbf{W}_{\text{IRNO}}$:

$$\mathbf{S}_{\text{IRNO}} = \mathbf{f}_i\, \mathbf{S}_{\text{SNNN}}. \tag{25}$$

8 Vertex Systems and Reconstruction

8.1 Vertex Systems

For any hull type $h \in \{\text{SNNN, CNO, IRNO}\}$, the vertex matrix is computed as

$$\mathbf{S}_h = \mathbf{W}_h^{\dagger}\,\mathbf{F} = \arg\min_{\mathbf{S}} \|\mathbf{F} - \mathbf{W}_h \mathbf{S}\|_F\,. \tag{26}$$

Each row $\mathbf{s}_j \in \mathbb{R}^n$ $(j = 1, \ldots, R')$ is a *vertex system*: a prototype state vector. The full trajectory is recovered as

$$\hat{\mathbf{F}} = \mathbf{W}_h\, \mathbf{S}_h. \tag{27}$$

8.2 Image-Space Projection

Given the observation matrix $\mathbf{C} \in \mathbb{R}^{P \times n}$ (mapping states to image pixels), each vertex system projects to a vertex *image*:

$$\mathbf{y}_j = \mathbf{C}\,\mathbf{s}_j \in \mathbb{R}^P. \tag{28}$$

The full image sequence is then

$$\hat{\mathbf{Y}}_k = \sum_{j=1}^{R'} w_{kj}\, \mathbf{y}_j, \tag{29}$$

which is a convex combination (for SNNN) of the R' vertex images, with time-varying weights $\{w_{kj}\}$.

8.3 Reconstruction Error

The reconstruction error is the Frobenius norm of the residual:

$$E_h = \|\mathbf{F} - \mathbf{W}_h \mathbf{S}_h\|_F \,. \tag{30}$$

Since SNNN is a linear transform of the truncated SVD basis, and since $\mathbf{S}_h$ is the optimal least-squares solution, we have

$$E_h = \left\|\mathbf{F} - \mathbf{U}_R \mathbf{U}_R^{\mathsf{T}} \mathbf{F}\right\|_F = \sqrt{\sum_{i=R+1}^{n} \sigma_i^2}, \tag{31}$$

i.e. the error equals the tail singular value energy regardless of the hull type. In practice, when $\mathbf{W}_h$ does not span exactly the column space of $\mathbf{U}_R$ (due to the augmentation), E_h can be slightly larger, but the difference is negligible.

9 Full Algorithm Summary

10 Connection to LMI-Based Controller Synthesis

The TP form $\mathbf{F}(p) = \sum_j w_j(p)\, \mathbf{s}_j$ becomes useful for control when $\mathbf{F}$ represents the parameter-dependent coefficient matrix of a linear system:

$$\mathbf{x}_{k+1} = \mathbf{A}(p_k)\, \mathbf{x}_k + \mathbf{B}(p_k)\, \mathbf{u}_k, \qquad \mathbf{A}(p_k) = \sum_{j=1}^{R'} w_j(p_k)\, \mathbf{A}_j. \tag{32}$$

Stability of (32) can be certified by finding a common quadratic Lyapunov function $\mathbf{P} \succ 0$ satisfying vertex LMIs:

$$\mathbf{A}_j^{\mathsf{T}} \mathbf{P} \mathbf{A}_j - \mathbf{P} \prec 0, \qquad j = 1, \ldots, R'. \tag{33}$$

This is tractable because only R' LMIs need to be checked rather than an infinite family. The SNNN property is essential: convexity of $\mathbf{W}$ guarantees that if (33) holds at every vertex, the Lyapunov condition holds everywhere.

10.1 Effect of Hull Type on Controller Design

Different hull types produce the same vertex count R' but affect the *size* of the vertex polytope and hence the conservatism of the LMI conditions:

Algorithm 1 TP Model Transformation of a State Trajectory

Require: Data matrix $\mathbf{F} \in \mathbb{R}^{N \times n}$, hull type h, threshold ε
Ensure: Weighting matrix $\mathbf{W}_h$, vertex matrix $\mathbf{S}_h$, error E_h

$\mathbf{U}, \mathbf{\Sigma}, \mathbf{V}^{\mathsf{T}} \leftarrow \mathrm{SVD}(\mathbf{F})$
$R \leftarrow \#\{i : \sigma_i > \varepsilon\, \sigma_1\}$, $R \leftarrow \max(R, 2)$
$\mathbf{U}_R \leftarrow \mathbf{U}_{:,1:R}$
// Inner centering (Section 4)
$\hat{\boldsymbol{u}} \leftarrow \frac{1}{N} \mathbf{U}_R^{\mathsf{T}} \mathbf{1}_N$
$\tilde{\mathbf{U}} \leftarrow \mathbf{U}_R - \mathbf{1}_N \hat{\boldsymbol{u}}^{\mathsf{T}}$
$\mathbf{U}_u, \mathbf{S}_u, \mathbf{V}_u \leftarrow \mathrm{SVD}(\tilde{\mathbf{U}})$, retain n_s non-negligible singular values
if $h = \mathrm{SNNN}$ **then**
    $\mathbf{W}, \mathbf{V}_1 \leftarrow \mathrm{SNNN}(\mathbf{U}_u)$ (Section 5)
else if $h = \mathrm{CNO}$ **then**
    $\mathbf{W}_s, \mathbf{V}_1 \leftarrow \mathrm{SNNN}(\mathbf{U}_u)$
    $\mathbf{W}, \mathbf{V}_2 \leftarrow \mathrm{CNO}(\mathbf{W}_s)$
    $\mathbf{V}_1 \leftarrow \mathbf{V}_2 \mathbf{V}_1$ (Section 6)
else if $h = \mathrm{IRNO}$ **then**
    $\mathbf{W}_s, \mathbf{V}_1 \leftarrow \mathrm{SNNN}(\mathbf{U}_u)$
    $\mathbf{W}, \mathbf{V}_2 \leftarrow \mathrm{IRNO}(\mathbf{W}_s)$
    $\mathbf{V}_1 \leftarrow \mathbf{V}_2 \mathbf{V}_1$ (Section 7)
end if
// Back-transform vertex matrix to original space
if $n_s < R$ **then**
    $\mathbf{V}_1 \leftarrow \mathbf{V}_1 \,\mathrm{diag}(\mathbf{S}_u)\, \mathbf{V}_u^{\mathsf{T}} + \mathbf{1}_{R'} \hat{\boldsymbol{u}}^{\mathsf{T}}$
end if
$\mathbf{W}_h \leftarrow \mathbf{W}$
$\mathbf{S}_h \leftarrow \mathbf{W}_h^{\dagger}\, \mathbf{F}$ (least squares, eq. (26))
$E_h \leftarrow \|\mathbf{F} - \mathbf{W}_h \mathbf{S}_h\|_F$

Hull	Vertex polytope	LMI conservatism
SNNN	Largest (standard simplex)	Most conservative
IRNO	Intermediate	Intermediate
CNO	Smallest (most balanced)	Least conservative

CNO produces the tightest enclosure of the data trajectory, which translates to the least conservative LMI conditions and typically the best achievable controller performance.

11 Application to Cortical Dynamics

In the cortical system setting:

- $\mathbf{F} = \mathbf{X} \in \mathbb{R}^{(T+1)\times n}$ is the state trajectory of the cortical network (n areas, $T+1$ time steps).
- The TP decomposition extracts R' *cortical activation modes* (vertex systems $\mathbf{s}_j \in \mathbb{R}^n$).
- The weighting functions $w_j(k)$ describe *when* and *how strongly* each mode dominates the cortical state.
- Each vertex image $\mathbf{y}_j = \mathbf{C}\mathbf{s}_j \in \mathbb{R}^{100\times 100}$ is a spatial activation map—interpretable as a distinct cortical "motif" (e.g. visual-dominated, somatosensory-dominated, resting-state).
- The reconstruction $\hat{\mathbf{Y}}_k = \sum_j w_j(k)\,\mathbf{y}_j$ represents the cortical image at time k as a blending of the discovered motifs.

This gives a compact, interpretable decomposition of the cortical dynamics: instead of n correlated time-series, we have $R' \leq n+1$ canonical spatial patterns and their time-varying mixture coefficients.

References

[1] P. Baranyi, *TP model transformation as a way to LMI-based controller design*, IEEE Transactions on Industrial Electronics, 51(2):387–400, 2004.

[2] P. Baranyi, Y. Yam, P. Várlaki, *Tensor Product Model Transformation in Polytopic Model-Based Control*, CRC Press, 2013.

[3] L. Szeidl, P. Várlaki, *HOSVD-based canonical form for TP models*, International Journal of Advanced Intelligence Paradigms, 2010.